ANNEX F

TECHNICAL DOCUMENTATION & SPECIFICATIONS

F.3 API ENDPOINT SPECIFICATIONS

Source review date: 10 November 2026

F.3.1 API Overview

ParSU e-Feedback exposes backend operations through Next.js App Router Route Handlers located under src/app/api. The source currently contains 68 route.ts handler files and 98 exported HTTP method handlers, including the GET and POST methods delegated to Auth.js. Dynamic route segments are shown in this annex as {id} or {attachmentId}; the corresponding Next.js folders use bracket notation, such as [id].

Handlers execute server-side in Node.js, parse URL parameters or JSON/form-data requests, apply authentication and resource-level authorization, connect to MongoDB through Mongoose, and return JSON with NextResponse.json. File operations use Cloudflare R2 through the S3-compatible SDK and Resend is called by email workflows. Auth.js is mounted at /api/auth/[...nextauth]; registration, code verification, and resend are separate Route Handlers.

The request guard `auth()` validates the Auth.js session against the current user record, active status, token version, and current authorization claims. Route handlers additionally enforce role, ownership, office, assignment, and head-designation rules. The edge proxy provides a coarse route-level filter for most application routes; route-level checks remain decisive. Password recovery and logout are implemented as Next.js Server Actions, not REST endpoints, and are identified separately in F.3.2.

PROJECT SOURCE TREE

Next.js API Route Directory

Live file listing generated from `src/app/api` · 68 route handler files

src/app/api/

```
├ admin/audit-logs/route.ts
├ admin/categories/[id]/route.ts
├ admin/categories/route.ts
├ admin/complaints/[id]/content/route.ts
├ admin/complaints/[id]/route.ts
├ admin/notifications/broadcast/route.ts
├ admin/offices/[id]/head/route.ts
├ admin/offices/[id]/route.ts
├ admin/offices/route.ts
├ admin/routing-rules/[id]/route.ts
├ admin/routing-rules/route.ts
├ admin/settings/route.ts
├ admin/sla-calendar/recalculate/route.ts
├ admin/sla-rules/[id]/route.ts
├ admin/sla-rules/route.ts
├ admin/users/[id]/route.ts
├ admin/users/route.ts
├ analytics/trends/route.ts
├ archive-requests/[id]/route.ts
├ archive-requests/route.ts
├ auth/[...nextauth]/route.ts
├ auth/register/route.ts
├ auth/resend-verification/route.ts
├ auth/verify-email/route.ts
├ categories/route.ts
├ colleges/route.ts
├ complaints/[id]/archive-request/route.ts
├ complaints/[id]/archive/route.ts
├ complaints/[id]/assign/route.ts
├ complaints/[id]/attachments/[attachmentId]/download/route.ts
├ complaints/[id]/attachments/route.ts
├ complaints/[id]/attachments/upload/route.ts
├ complaints/[id]/close-without-rating/route.ts
├ complaints/[id]/escalate/route.ts
├ complaints/[id]/forwarding/route.ts
├ complaints/[id]/information-request/route.ts
├ complaints/[id]/information-response/route.ts
├ complaints/[id]/notes/route.ts
├ complaints/[id]/rate/route.ts
├ complaints/[id]/reassign-office/route.ts
├ complaints/[id]/release/route.ts
├ complaints/[id]/reopen/route.ts
├ complaints/[id]/resolution-requests/route.ts
├ complaints/[id]/route.ts
├ complaints/route.ts
├ cron/sla-check/route.ts
├ dashboard/route.ts
├ dev/seed/route.ts
├ dev/test-email/route.ts
├ feedback/[id]/route.ts
├ feedback/route.ts
├ forwarding-requests/[id]/route.ts
├ forwarding-requests/route.ts
├ me/email-change/confirm/route.ts
├ me/email-change/route.ts
├ me/route.ts
├ notifications/[id]/read/route.ts
├ notifications/read-all/route.ts
├ notifications/route.ts
├ offices/[id]/staff/route.ts
├ offices/route.ts
├ reports/[id]/download/route.ts
├ reports/[id]/print/route.ts
├ reports/[id]/route.ts
├ reports/export/route.ts
├ resolution-requests/[id]/route.ts
├ resolution-requests/route.ts
├ uploads/presign/route.ts
```

Figure F.4. Next.js API Route Directory. Generated from the actual src/app/api file tree.

F.3.2 API Endpoint Inventory

Table F.9 lists each implemented Route Handler method separately. Dynamic URL parameters are shown in braces. Access levels describe the handler’s server-side checks; they are not inferred from the UI.

Table F.9. API Endpoint Inventory

HTTP Method	API Endpoint	Description	Access Level
Authentication and Student Registration			
GET	/api/auth/[...nextauth]	Auth.js GET protocol handler (session/CSRF and related authentication endpoints).	Auth.js protocol; session requirements vary by action.
POST	/api/auth/[...nextauth]	Auth.js POST protocol handler (credential sign-in/sign-out and related authentication actions).	Auth.js protocol; credentials/session requirements vary by action.
POST	/api/auth/register	Register a student account and initiate email verification.	Public; student email/domain checks, verification token/code checks, and rate limits apply.
POST	/api/auth/resend-verification	Resend a pending student verification code subject to cooldown and rate limits.	Public; student email/domain checks, verification token/code checks, and rate limits apply.
POST	/api/auth/verify-email	Verify a student email address using its six-digit code.	Public; student email/domain checks, verification token/code checks, and rate limits apply.
User Management			
GET	/api/admin/users	List users with pagination and role filtering; password hashes are excluded.	Administrator only (scoped-role guard mismatch; see F.3.4).
POST	/api/admin/users	Create a role-scoped user account; student verification email is attempted when applicable.	Administrator only (scoped-role guard mismatch; see F.3.4).
DELETE	/api/admin/users/{id}	Delete a user subject to administrator/self-deletion safeguards.	Administrator only (scoped-role guard mismatch; see F.3.4).
GET	/api/admin/users/{id}	Retrieve one user while excluding password hash data.	Administrator only (scoped-role guard mismatch; see F.3.4).
PATCH	/api/admin/users/{id}	Update user profile/role/office/college/status/password or force-logout state.	Administrator only (scoped-role guard mismatch; see F.3.4).
GET	/api/me	Return the signed-in user's own profile and pending email-change metadata.	Authenticated user; own profile only.
PATCH	/api/me	Update allowed own-profile fields and/or password after current-password checks.	Authenticated user; own editable profile fields only.
POST	/api/me/email-change	Request a login-email change and send a confirmation link.	Authenticated user; current-password check and rate limit.
POST	/api/me/email-change/confirm	Confirm an email change using a single-use token.	No session; valid single-use token.
Complaint Submission and Retrieval			
GET	/api/complaints	List complaints filtered by student ownership, office assignment, or admin scope.	Authenticated; results filtered by owner, office, or admin scope.
POST	/api/complaints	Create a student complaint, assign its ticket and SLA dates, and route it initially to active OSAS.	Authenticated student only.
GET	/api/complaints/{id}	Retrieve an accessible complaint with a role-filtered timeline.	Authenticated; complaint ownership/current office/admin-scope checks.
PATCH	/api/complaints/{id}	Edit/withdraw an owner's unassigned complaint or update status as its assigned handler.	Student owner before pickup, or assigned complaint handler.
Complaint Management and Status Updates			
GET	/api/archive-requests	List pending archive requests for the signed-in office head.	Designated active office head.
PATCH	/api/archive-requests/{id}	Approve/reject a pending archive request for the responsible office.	Designated head of the request's office.
PATCH	/api/complaints/{id}/archive	Archive or restore a complaint after office-head/administrator supervision checks.	Administrator; OSAS Head for OSAS-owned complaint.
GET	/api/complaints/{id}/archive-request	Retrieve the complaint's archive request after complaint access checks.	Authenticated internal complaint viewer.
POST	/api/complaints/{id}/archive-request	Submit an archive request for review by the responsible office head.	Assigned office_staff on OSAS complaint.
POST	/api/complaints/{id}/close-without-rating	Close an eligible resolved complaint without a student rating.	Authenticated student owner of resolved complaint.
POST	/api/complaints/{id}/escalate	Submit a manual escalation with a required reason.	Administrator or assigned OSAS handler on OSAS complaint.
POST	/api/complaints/{id}/information-request	Create a request for additional student information.	Assigned handler for current complaint office.
POST	/api/complaints/{id}/information-response	Submit the student's response to an open information request.	Authenticated student owner; complaint awaits information.
GET	/api/complaints/{id}/notes	List internal notes for an authorized complaint handler.	Authenticated internal complaint viewer.
POST	/api/complaints/{id}/notes	Add an internal note to an accessible complaint.	Authenticated internal user with note-write authority.
POST	/api/complaints/{id}/rate	Submit a 1–5 satisfaction rating and optional comment for a resolved complaint.	Authenticated student owner of resolved complaint.
POST	/api/complaints/{id}/reopen	Reopen an eligible complaint with a required reason and audit/timeline event.	Administrator or OSAS Head with complaint access.
GET	/api/complaints/{id}/resolution-requests	List resolution requests for an internally accessible complaint.	Authenticated internal complaint viewer.
POST	/api/complaints/{id}/resolution-requests	Submit a resolution request from the assigned handler for office-head review.	Assigned handler; office_staff/OSAS/VPAA/VPAF.
GET	/api/resolution-requests	List pending resolution requests for the signed-in office head.	Designated active office head.
PATCH	/api/resolution-requests/{id}	Approve/reject a pending resolution request; approval resolves the complaint.	Designated head of the request's office.
Complaint Assignment and Forwarding			
POST	/api/complaints/{id}/assign	Assign/reassign a complaint to an eligible staff handler through office controls.	Responsible office head; administrator override for OSAS.
GET	/api/complaints/{id}/forwarding	List forwarding history for an accessible complaint.	Authenticated user with complaint access.
POST	/api/complaints/{id}/forwarding	Create an OSAS forwarding request or directly forward as OSAS Head.	Assigned OSAS handler may request; OSAS Head may forward.

POST	/api/complaints/{id}/reassign-office	Reject the legacy reassignment request and direct callers to controlled forwarding.	Authenticated; handler enforces role/resource scope.
POST	/api/complaints/{id}/release	Release the handler assignment with a required reason.	Assigned OSAS/office_staff handler.
GET	/api/forwarding-requests	List authorized forwarding-request history, optionally filtered by status.	Administrator, OSAS Head, or OSAS requester.
PATCH	/api/forwarding-requests/{id}	Approve/reject a pending OSAS forwarding request as OSAS Head.	OSAS Head only.
SLA Monitoring			
GET	/api/admin/sla-calendar/recalculate	Preview recalculated complaint SLA dates and return a freshness token.	Administrator only.
POST	/api/admin/sla-calendar/recalculate	Apply the SLA date plan only when the submitted preview token is current.	Administrator only.
GET	/api/admin/sla-rules	List SLA rules.	Administrator only.
POST	/api/admin/sla-rules	Create an SLA rule for a category/priority combination.	Administrator only.
DELETE	/api/admin/sla-rules/{id}	Delete an SLA rule.	Administrator only.
PATCH	/api/admin/sla-rules/{id}	Update an SLA rule after category and threshold validation.	Administrator only.
GET	/api/cron/sla-check	Run the secret-protected scheduled SLA warning and escalation scan.	Shared cron secret required.
POST	/api/cron/sla-check	Run the secret-protected SLA scan for QStash/manual callers.	Shared cron secret required.
General Student Feedback			
GET	/api/feedback	List the signed-in student's non-archived feedback.	Authenticated student; own feedback only.
POST	/api/feedback	Submit standalone general feedback as the signed-in student.	Authenticated student only.
DELETE	/api/feedback/{id}	Soft-archive the signed-in student's own feedback record.	Authenticated student; own feedback only.
PATCH	/api/feedback/{id}	Update the signed-in student's own feedback record.	Authenticated student; own feedback only.
File Attachments			
GET	/api/complaints/{id}/attachments	List attachments for an accessible complaint.	Authenticated user with complaint access.
POST	/api/complaints/{id}/attachments	Register an attachment record on an accessible complaint.	Authenticated user allowed to attach to complaint.
POST	/api/complaints/{id}/attachments/upload	Upload a student information-response file to R2 and register the attachment.	Authenticated student complaint owner.
GET	/api/complaints/{id}/attachments/{attachmentId}/download	Download an authorized attachment from Cloudflare R2.	Authenticated user with complaint access.
POST	/api/uploads/presign	Validate an attachment request and issue a short-lived Cloudflare R2 upload URL.	Authenticated user allowed to attach to complaint.
Notifications			
POST	/api/admin/notifications/broadcast	Create a broadcast notification for all users, a role, or an office.	Administrator only.
GET	/api/notifications	List the caller's visible notifications and unread count.	Authenticated; own recipient and accessible-complaint scope.
PATCH	/api/notifications/read-all	Mark all of the caller's unread notifications as read.	Authenticated; own notifications only.
PATCH	/api/notifications/{id}/read	Mark one owned, in-scope notification as read.	Authenticated; own in-scope notification only.
Institutional Analytics			
GET	/api/analytics/trends	Return complaint summary, monthly trends, and category, priority, domain, and college breakdowns within the caller's scope.	Authenticated administrator/OSAS/VPAA/VPAF/office_staff.
GET	/api/dashboard	Return role-scoped dashboard counts and complaint lists.	Authenticated administrator/OSAS/VPAA/VPAF/office_staff.
Reports and Exporting			
GET	/api/reports/export	List up to 50 generated reports created by the signed-in user.	Authenticated; own generated reports only.
POST	/api/reports/export	Generate a CSV, Excel, or PDF report using caller-scoped complaint data.	Authenticated non-student; report data scoped by role/office.
DELETE	/api/reports/{id}	Delete an authorized generated report.	Authenticated; handler enforces role/resource scope.
GET	/api/reports/{id}/download	Download a ready report from Cloudflare R2 after access checks.	Authenticated; handler enforces role/resource scope.
GET	/api/reports/{id}/print	Render an authorized report for printing.	Authenticated; handler enforces role/resource scope.
Administrative Configuration			
GET	/api/admin/audit-logs	Search and paginate system audit records by entity, action, actor, or date range.	Administrator only.
GET	/api/admin/categories	List all categories, including inactive/draft categories, for administrators.	Administrator only.
POST	/api/admin/categories	Create a category and its initial active OSAS routing rule; the category is created inactive pending SLA setup.	Administrator only.
DELETE	/api/admin/categories/{id}	Delete a category when dependent records permit it.	Administrator only.
GET	/api/admin/categories/{id}	Retrieve one category by ID.	Administrator only.
PATCH	/api/admin/categories/{id}	Update category fields after schema and routing checks.	Administrator only.
PATCH	/api/admin/complaints/{id}	Archive or restore the complaint with reason and audit entry.	Administrator only (scoped-role guard mismatch; see F.3.4).
PATCH	/api/admin/complaints/{id}/content	Correct complaint title/description with a reason and audit trail.	Administrator only.
GET	/api/admin/offices	List offices available within the caller's permitted administrative scope.	Administrator only (scoped-role guard mismatch; see F.3.4).
POST	/api/admin/offices	Create an office after office-type, code, and caller-scope validation.	Administrator only (scoped-role guard mismatch; see F.3.4).

DELETE	/api/admin/offices/{id}	Delete an office when it has no assigned users or dependent records.	Administrator only (scoped-role guard mismatch; see F.3.4).
GET	/api/admin/offices/{id}	Retrieve an office record after scope authorization.	Administrator only (scoped-role guard mismatch; see F.3.4).
PATCH	/api/admin/offices/{id}	Update office metadata after scope and hierarchy validation.	Administrator only (scoped-role guard mismatch; see F.3.4).
DELETE	/api/admin/offices/{id}/head	Remove the office-head designation.	Administrator only.
PATCH	/api/admin/offices/{id}/head	Assign or replace an eligible active user as office head.	Administrator only.
POST	/api/admin/offices/{id}/head	Create an eligible user account and assign it as the office head.	Administrator only.
GET	/api/admin/routing-rules	List routing-rule records.	Administrator only.
POST	/api/admin/routing-rules	Create a category routing rule with validated target and conditions.	Administrator only.
DELETE	/api/admin/routing-rules/{id}	Delete a routing-rule record.	Administrator only.
PATCH	/api/admin/routing-rules/{id}	Update a routing rule while preventing duplicate active category rules.	Administrator only.
GET	/api/admin/settings	Return persisted application settings.	Administrator only.
PATCH	/api/admin/settings	Update validated runtime settings, including upload policy and SLA holidays.	Administrator only.
GET	/api/categories	List active complaint categories with minimal fields for signed-in users.	Any authenticated user; response contains active/minimal fields.
GET	/api/colleges	List active college offices with name/code fields for registration.	Public, read-only.
GET	/api/offices	List active offices with name/code/type fields for authenticated users.	Any authenticated user; response contains active/minimal fields.
GET	/api/offices/{id}/staff	List eligible active staff for an office the caller is authorized to manage.	Responsible head for that office.
Development and Operations			
GET	/api/dev/seed	Run the development seed operation when environment and secret checks pass.	Non-production and development secret required.
POST	/api/dev/test-email	Send a verification-format test email to the configured test recipient in non-production.	Non-production and development secret required.

Inventory coverage: 98 HTTP method handlers across 68 route files. The auth catch-all delegates GET/POST to Auth.js. No additional root-level app/api directory or backend route directory was found.

Server Actions (not REST endpoints): forgotPasswordAction and resetPasswordAction in src/features/auth/actions.ts; logoutAction in src/features/auth/actions/logout.action.ts. Password recovery validates FormData, applies the IP/email rate limiter, returns a generic response that does not reveal account existence, and resets a password only with a valid token. Logout calls Auth.js signOut and redirects to /login.

READ-ONLY LOCAL API CHECK

Sample API Request and Response

Local development · sanitized output · Tue, 10 Nov 2026 15:17:13 GMT

GET http://localhost:3000/api/colleges
HTTP 200


```
{
  "colleges": [
    {
      "_id": "[redacted]",
      "name": "College of Arts and Humanities",
      "code": "CAH"
    },
    {
      "_id": "[redacted]",
      "name": "College of Business & Management",
      "code": "CBM"
    },
    {
      "_id": "[redacted]",
      "name": "College of Education",
      "code": "CED"
    },
    {
      "_id": "[redacted]",
      "name": "College of Engineering & Computational Sciences",
      "code": "CECS"
    },
    {
      "_id": "[redacted]",
      "name": "College of Science",
      "code": "COS"
    }
  ]
}
```


Response came from the running local application. Database identifiers are redacted; no records were modified.

Figure F.5. Sample API Request and Response. Read-only GET request captured from the running local development application; database identifiers are redacted.

F.3.3 Detailed API Endpoint Specifications

The following specifications cover major implemented flows. Request and response examples are illustrative and sanitized; the success/error behavior is taken from the route handler and its imported schemas/services. Full method coverage remains in Table F.9.

Authentication and registration

Endpoint Name: POST /api/auth/register

Specification	Actual Implementation
Endpoint URL	POST /api/auth/register
Source File	src/app/api/auth/register/route.ts
Authentication / Authorized Roles	No prior session; public student self-registration.
Request Parameters / Body	JSON: firstName, lastName, email (@parsu.edu.ph), studentNumber, collegeId, password, confirmPassword.
Success Response	201 {message}; account is created unverified and a code is sent.
Error Responses	400 invalid JSON/validation/college; 409 duplicate email/student number; 429 rate-limited (Retry-After); 503 limiter/email delivery unavailable.

Endpoint Name: POST /api/auth/verify-email

Specification	Actual Implementation
Endpoint URL	POST /api/auth/verify-email
Source File	src/app/api/auth/verify-email/route.ts
Authentication / Authorized Roles	Public; six-digit code verification for a ParSU student email.
Request Parameters / Body	JSON: email, code (exactly six digits).
Success Response	200 {message: 'Email verified. You can now sign in.'}.
Error Responses	400 invalid body/code; 429 rate limit; 503 rate limiter unavailable.

Endpoint Name: POST /api/auth/resend-verification

Specification	Actual Implementation
Endpoint URL	POST /api/auth/resend-verification
Source File	src/app/api/auth/resend-verification/route.ts
Authentication / Authorized Roles	Public; pending student account only, with cooldown and rate limits.
Request Parameters / Body	JSON: email.
Success Response	200 {message}; Retry-After: 60.
Error Responses	400 invalid body; 429 too many requests with Retry-After; 503 delivery/rate-limit error.

Endpoint Name: GET, POST /api/auth/[...nextauth]

Specification	Actual Implementation
Endpoint URL	GET, POST /api/auth/[...nextauth]
Source File	src/app/api/auth/[...nextauth]/route.ts
Authentication / Authorized Roles	Auth.js handler; endpoint actions determine whether a session is needed.
Request Parameters / Body	Auth.js protocol parameters/form fields; credential sign-in uses email and password.
Success Response	Auth.js-defined JSON/redirect/session responses.
Error Responses	Auth.js-defined authentication and protocol errors.

Complaint submission and status

Endpoint Name: POST /api/complaints

Specification	Actual Implementation
Endpoint URL	POST /api/complaints
Source File	src/app/api/complaints/route.ts
Authentication / Authorized Roles	Authenticated student only.
Request Parameters / Body	JSON: categoryRef, title (5–200 chars), description (at least 20 chars). Category must be active; matching SLA rule required.
Success Response	201 {complaint}; also records timeline/audit and notifications.
Error Responses	401 unauthenticated; 403 non-student; 400 invalid body/category; 422 OSAS unavailable or no SLA rule.

Endpoint Name: GET /api/complaints

Specification	Actual Implementation
Endpoint URL	GET /api/complaints
Source File	src/app/api/complaints/route.ts
Authentication / Authorized Roles	Authenticated; student, office_staff, administrator, OSAS, VPAA, VPAF; data filtered per user/office/scope.
Request Parameters / Body	Query: page, limit (max 100, default 20), status, q (title/ticket search).
Success Response	200 {complaints,total,page,limit}.
Error Responses	401 no session; 403 unsupported role.

Endpoint Name: GET /api/complaints/{id}

Specification	Actual Implementation
Endpoint URL	GET /api/complaints/{id}
Source File	src/app/api/complaints/[id]/route.ts
Authentication / Authorized Roles	Authenticated with complaint ownership/current-office/admin-scope access.
Request Parameters / Body	Path: id.
Success Response	200 {complaint,timeline}; student timeline omits internal resolution-request/rejection events and student actor identities are redacted for staff.
Error Responses	401 unauthenticated; 403 not authorized; 404 not found.

Endpoint Name: PATCH /api/complaints/{id}

Specification	Actual Implementation
Endpoint URL	PATCH /api/complaints/{id}
Source File	src/app/api/complaints/[id]/route.ts
Authentication / Authorized Roles	Authenticated; owner-student can edit/withdraw only while submitted; assigned handler can update permitted statuses. A non-head handler cannot mark a complaint resolved directly.
Request Parameters / Body	JSON for student: title and/or description, or status='withdrawn'. Handler: status plus optional message/resolutionSummary.

Success Response	200 {complaint}; timeline/audit/notification side effects on applicable mutations.
Error Responses	400 invalid body/transition or complaint already picked up; 401; 403; 404. Generic status route rejects direct assigned/escalated/closed/pending-information transitions with workflow guidance.

OSAS forwarding workflow

Endpoint Name: POST /api/complaints/{id}/forwarding

Specification	Actual Implementation
Endpoint URL	POST /api/complaints/{id}/forwarding
Source File	src/app/api/complaints/[id]/forwarding/route.ts
Authentication / Authorized Roles	Authenticated OSAS/office_staff. Assigned OSAS handler may request; OSAS Head may directly forward.
Request Parameters / Body	JSON: destination ('vpaa'|'vpaf'), reason (1–4000 chars), optional direct boolean.
Success Response	201 {request}; direct head action creates and approves in one transaction, then transfers office/handler and records timeline/audit/notifications.
Error Responses	400 schema/rejection reason; 401; 403 role/assignment/head restriction; 404; 409 invalid state/pending archive/config; 500 generic persistence failure.

Endpoint Name: GET /api/complaints/{id}/forwarding

Specification	Actual Implementation
Endpoint URL	GET /api/complaints/{id}/forwarding
Source File	src/app/api/complaints/[id]/forwarding/route.ts
Authentication / Authorized Roles	Authenticated with complaint access.
Request Parameters / Body	Path: id.
Success Response	200 {requests}, newest first, with requester/reviewer identity fields populated.
Error Responses	401; 403; 404.

Endpoint Name: GET /api/forwarding-requests

Specification	Actual Implementation
Endpoint URL	GET /api/forwarding-requests
Source File	src/app/api/forwarding-requests/route.ts
Authentication / Authorized Roles	Authenticated administrator, OSAS Head, or OSAS requester; data is filtered by requester/head.
Request Parameters / Body	Query: optional status=pending|approved|rejected.
Success Response	200 {requests}, maximum 100, newest first.
Error Responses	400 invalid status; 401; 403 unauthorized history scope.

Endpoint Name: PATCH /api/forwarding-requests/{id}

Specification	Actual Implementation
Endpoint URL	PATCH /api/forwarding-requests/{id}
Source File	src/app/api/forwarding-requests/[id]/route.ts
Authentication / Authorized Roles	Authenticated OSAS Office Head only.
Request Parameters / Body	JSON: action ('approve'|'reject'), reason (optional, required for rejection).
Success Response	200 {request}; approval transfers complaint to active OVPAA/OVPAF and designated receiving handler.
Error Responses	400 invalid decision/rejection reason; 401; 403; 404; 409 already decided/ineligible/configuration; 500 generic failure.

Resolution approval workflow

Endpoint Name: POST /api/complaints/{id}/resolution-requests

Specification	Actual Implementation
Endpoint URL	POST /api/complaints/{id}/resolution-requests
Source File	src/app/api/complaints/[id]/resolution-requests/route.ts
Authentication / Authorized Roles	Authenticated assigned handler with supported role office_staff|osas|vpaa|vpaf; complaint processing access required.
Request Parameters / Body	JSON: optional resolutionSummary (max 4000) and message (max 2000). Office head must be configured; complaint must be active and transitionable to resolved.
Success Response	201 {request}; request, timeline, audit record, and head notification are created.
Error Responses	400 invalid body/not ready/unassigned; 401; 403 wrong role/handler/archive; 404; 409 missing head or pending duplicate.

Endpoint Name: GET /api/resolution-requests

Specification	Actual Implementation

Endpoint URL	GET /api/resolution-requests
Source File	src/app/api/resolution-requests/route.ts
Authentication / Authorized Roles	Authenticated designated active head of the relevant office (authority is Office.headUserRef, not a dedicated role).
Request Parameters / Body	No query parameters.
Success Response	200 {requests}; pending requests for the caller's active head-designated office(s).
Error Responses	401; 403 not an active office head.

Endpoint Name: PATCH /api/resolution-requests/{id}

Specification	Actual Implementation
Endpoint URL	PATCH /api/resolution-requests/{id}
Source File	src/app/api/resolution-requests/[id]/route.ts
Authentication / Authorized Roles	Authenticated responsible office head; verifies office designation and complaint scope/state.
Request Parameters / Body	JSON: action ('approve'|'reject'); rejectionReason required for reject (max 2000).
Success Response	200 {request,complaint}; approval resolves complaint; rejection preserves current complaint status. Both create timeline/audit/notifications.
Error Responses	400 schema; 401; 403; 404; 409 duplicate decision or changed complaint state; 500 persistence verification failure.

Attachments and R2

Endpoint Name: POST /api/uploads/presign

Specification	Actual Implementation
Endpoint URL	POST /api/uploads/presign
Source File	src/app/api/uploads/presign/route.ts
Authentication / Authorized Roles	Authenticated user permitted to attach to referenced complaint.
Request Parameters / Body	JSON: complaintId, fileName, mimeType, sizeBytes. MIME and maximum size come from settings.
Success Response	200 {uploadUrl,fileUrl,objectKey}; signed URL expires after 300 seconds.
Error Responses	400 validation/type/size; 401; 403 inaccessible/read-only complaint; 404 complaint; 501 R2 not configured.

Endpoint Name: POST /api/complaints/{id}/attachments/upload

Specification	Actual Implementation
Endpoint URL	POST /api/complaints/{id}/attachments/upload
Source File	src/app/api/complaints/[id]/attachments/upload/route.ts
Authentication / Authorized Roles	Authenticated student owner only; complaint must be active and non-archived.
Request Parameters / Body	multipart/form-data field file; type/size checked against settings.
Success Response	201 {attachment}; file is uploaded to R2 and an attachment timeline/audit record is added.
Error Responses	400 missing/invalid/oversize file; 401; 403; 404; 501 R2 not configured.

Endpoint Name: GET /api/complaints/{id}/attachments/{attachmentId}/download

Specification	Actual Implementation
Endpoint URL	GET /api/complaints/{id}/attachments/{attachmentId}/download
Source File	src/app/api/complaints/[id]/attachments/[attachmentId]/download/route.ts
Authentication / Authorized Roles	Authenticated user with complaint access.
Request Parameters / Body	Path: complaint id and attachment id.
Success Response	200 file bytes with attachment content headers.
Error Responses	401; 403; 404; 501 storage not configured; 502 storage returned empty content.

Analytics, reports, and notifications

Endpoint Name: GET /api/analytics/trends

Specification	Actual Implementation
Endpoint URL	GET /api/analytics/trends
Source File	src/app/api/analytics/trends/route.ts
Authentication / Authorized Roles	Authenticated administrator, OSAS, VPAA, VPAF, or office_staff; scoped aggregates.
Request Parameters / Body	No query/body.
Success Response	200 {summary,trends,collegeComparison,categoryBreakdown,priorityBreakdown,domainBreakdown}; office_staff collegeComparison is empty.
Error Responses	401; 403 unsupported role.

Endpoint Name: GET /api/dashboard

Specification	Actual Implementation
Endpoint URL	GET /api/dashboard
Source File	src/app/api/dashboard/route.ts
Authentication / Authorized Roles	Authenticated administrator, OSAS, VPAA, VPAF, or office_staff.
Request Parameters / Body	No query/body.
Success Response	200 role-scoped dashboard data from getDashboardData.
Error Responses	401; 403 student/unsupported role.

Endpoint Name: POST /api/reports/export

Specification	Actual Implementation
Endpoint URL	POST /api/reports/export
Source File	src/app/api/reports/export/route.ts
Authentication / Authorized Roles	Authenticated non-student; report rows are scoped by role and office.
Request Parameters / Body	JSON: format csv|excel|pdf; optional reportType and filters (office/college/domain/priority/SLA/category/date/status).
Success Response	200 {report}; report status becomes ready and includes downloadUrl after R2 upload.
Error Responses	400 Zod validation; 401; 403 student; 500 report generation failure.

Endpoint Name: GET /api/notifications

Specification	Actual Implementation
Endpoint URL	GET /api/notifications
Source File	src/app/api/notifications/route.ts
Authentication / Authorized Roles	Authenticated; recipient and related-complaint visibility are enforced.
Request Parameters / Body	Query: unreadOnly=true, limit (max 100, default 20).
Success Response	200 {notifications,unreadCount}.
Error Responses	401 unauthenticated.

Administrative configuration

Endpoint Name: GET, POST /api/admin/users

Specification	Actual Implementation
Endpoint URL	GET, POST /api/admin/users
Source File	src/app/api/admin/users/route.ts
Authentication / Authorized Roles	The source calls requireScopedAdmin; actual helper currently admits administrator only (see F.3.4).
Request Parameters / Body	GET query page,limit (max 2000),role. POST JSON: employeeOrStudentId,firstName,lastName,email,password,role and optional officeRef,collegeRef,isActive.
Success Response	GET 200 {users,total,page,limit}. POST 201 {user} (passwordHash removed); student verification email may be sent.
Error Responses	400 validation; 401; 403 role/scope conflict; 409 duplicate; 503 verification email delivery.

Endpoint Name: GET, PATCH /api/admin/settings

Specification	Actual Implementation
Endpoint URL	GET, PATCH /api/admin/settings
Source File	src/app/api/admin/settings/route.ts
Authentication / Authorized Roles	Authenticated administrator only via requireAdmin.
Request Parameters / Body	PATCH JSON supports ticketNumberPrefix,slaHolidays,uploadMaxFileSizeMb,uploadAllowedMimeTypes,authMaxFailedLoginAttempts,authLockoutDurationMinutes.
Success Response	GET 200 settings; PATCH 200 updated settings.
Error Responses	401; 403; 400 Zod validation.

Endpoint Name: GET, POST /api/admin/sla-calendar/recalculate

Specification	Actual Implementation
Endpoint URL	GET, POST /api/admin/sla-calendar/recalculate
Source File	src/app/api/admin/sla-calendar/recalculate/route.ts
Authentication / Authorized Roles	Authenticated administrator only.

Request Parameters / Body	GET no body. POST JSON: token from current preview.
Success Response	GET 200 {token,count,skipped,sample}; POST 200 recalculation result and audit record.
Error Responses	401; 403; 409 stale preview token.

Scheduled SLA operation

Endpoint Name: GET, POST /api/cron/sla-check

Specification	Actual Implementation
Endpoint URL	GET, POST /api/cron/sla-check
Source File	src/app/api/cron/sla-check/route.ts
Authentication / Authorized Roles	No user session; shared CRON_SECRET required as Bearer token or x-cron-secret.
Request Parameters / Body	No body required; GET is Vercel Cron-compatible, POST is QStash/manual-compatible.
Success Response	200 {checked,escalated,warned}; checks active OSAS complaints, sends warnings/escalates where due, updates last-check setting.
Error Responses	401 secret mismatch; 503 active OSAS office unavailable.

Illustrative sanitized JSON examples (not production records):

Complaint submission request: `{"categoryRef":"<category-id>","title":"Projector unavailable","description":"The classroom projector has not worked since Monday."}`

Forwarding request body: `{"destination":"vpaa","reason":"The issue requires academic policy review."}`

Resolution decision body: `{"action":"reject","rejectionReason":"Please include the completed remediation details."}`

ACTUAL SOURCE SNAPSHOT

API Authentication and Authorization

src/features/forwarding/forwarding.service.ts · OSAS role, assignment, direct-forward, and head-decision checks

src/features/forwarding/forwarding.service.ts

```
53    const creating = Boolean(input.complaintId);
54    const parsed = creating
55      ? requestSchema.safeParse(input.body)
56      : decisionSchema.safeParse(input.body);
57    if (!parsed.success)
58      throw new ForwardingError(parsed.error.issues[0]?.message ?? "Invalid forwarding request", 400);
59    if (!["osas", "office_staff"].includes(session.user.role))
60      throw new ForwardingError("Only OSAS may request or perform forwarding.", 403);
    ...
81        const osas = await Office.findOne({ code: "OSAS", isActive: true }).session(dbSession);
82        const head = Boolean(osas && refId(osas.headUserRef) === session.user.id);
83        if (!osas || refId(complaint.assignedOfficeRef) !== refId(osas._id))
84          throw new ForwardingError("Complaint is no longer under OSAS.", 403);
85        if (
86          (creating || data.action === "approve") &&
87          (complaint.isArchived || !active.includes(complaint.status))
88        )
89          throw new ForwardingError("Forwarding requires an active complaint.");
90        if (creating) {
91          if ((!head && !(await canProcessComplaint(session, complaint))) || (data.direct && !head))
92            throw new ForwardingError(
93              "Only the assigned OSAS handler may request forwarding; only the OSAS Head may forward.",
94              403,
95            );
    ...
152       const decision = creating ? (data.direct ? "approve" : null) : data.action;
153       if (decision) {
154         if (!head)
155           throw new ForwardingError(
156             "Only the OSAS Office Head may decide forwarding requests.",
157             403,
158           );
159         if (request.status !== "pending")
160           throw new ForwardingError("This forwarding request has already been decided.");
161         if (decision === "reject" && !data.reason)
162           throw new ForwardingError("A rejection reason is required.", 400);
163         request.status = decision === "approve" ? "approved" : "rejected";
164         request.reviewedByUserRef = session.user.id;
165         request.reviewedAt = new Date();
166         request.reviewReason = data.reason ?? "";
167         await request.save({ session: dbSession });
```

Figure F.6. API Authentication and Authorization Implementation. Forwarding authorization from src/features/forwarding/forwarding.service.ts.

ACTUAL SOURCE SNAPSHOT

Complaint Management API

src/app/api/complaints/route.ts · authentication, student validation, and OSAS routing

src/app/api/complaints/route.ts

```
27  export async function POST(req: NextRequest) {
28    const session = await auth();
29    if (!session?.user) {
30      return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
31    }
32    if (session.user.role !== "student") {
33      return NextResponse.json({ error: "Only students can file complaints" }, { status: 403 });
34    }
35
36    await connectToDatabase();
37
38    const body = await req.json();
39    const parsed = createComplaintSchema.safeParse(body);
40    if (!parsed.success) {
41      return NextResponse.json({ error: parsed.error.flatten() }, { status: 400 });
42    }
43
44    const category = await Category.findById(parsed.data.categoryRef);
45    if (!category || !category.isActive) {
46      return NextResponse.json({ error: "Invalid or inactive category" }, { status: 400 });
47    }
48
49    const student = await User.findById(session.user.id).lean();
50
51    // --- Resolve routing office (may throw RoutingError) ---
52    let resolvedOfficeRef: string;
53    try {
54      const result = await resolveRoutingOffice({
55        categoryId: category._id.toString(),
56        studentCollegeRef: student ? String((student as any).collegeRef ?? "") || null : null,
57        priority: category.defaultPriority,
58      });
59      resolvedOfficeRef = result.officeRef;
60    } catch (error) {
61      if (error instanceof RoutingError) {
62        return NextResponse.json({ error: error.message, code: error.code }, { status: 422 });
63      }
64      throw error;
65    }
```

Figure F.7. Complaint Management API Implementation. Source snapshot from src/app/api/complaints/route.ts.

F.3.4 API Authentication and Authorization

Auth.js provides the credential/session protocol at /api/auth/[...nextauth]. Student self-registration is limited by registerSchema to @parsu.edu.ph and requires first/last name, student number, college, password, and password confirmation. Code verification and resend are public endpoints but validate the email/code, enforce cooldowns and Upstash rate limits, and do not require a session. The application's `auth()` wrapper returns no valid session when the user is missing/inactive, tokenVersion differs, or session claims no longer match the database user.

Students may create complaints and access their own records. They can edit or withdraw only while the complaint remains submitted, respond to information requests, attach files to their complaints, rate a resolved complaint, and manage their own profile, feedback, notifications, and email change. The handlers enforce ownership by comparing the session user ID with the stored studentRef; UI visibility is not treated as authorization.

OSAS office_staff and OSAS-role users may process an OSAS complaint only when they are the assigned handler. Office heads are not a separate user role: the authority is derived from an active Office document whose headUserRef equals the signed-in user. This designation controls staff assignment, resolution approval, archive review, and OSAS forwarding decisions. Assignment management for OSAS is limited to the OSAS Head (or administrator override where coded). The generic complaint PATCH permits a designated head who is also the assigned handler to resolve directly; a non-head handler is directed to the separate resolution-request/approval workflow.

VPAA/VPAF accounts can see/process receiving-office complaints only after a formal forwarding marker, matching receiving office, and assigned handler are present. The forwarding service accepts requests from an assigned OSAS handler; only the OSAS Head may approve/reject or use `direct: true` to formally forward to VPAA/VPAF. Approval assigns the complaint to the designated receiving office head (or the sole active account when exactly one exists), updates ownership/status, and writes timeline, audit, and notifications in a transaction. Administrators have broad

complaint access, but the forwarding service's role check explicitly accepts OSAS/office_staff only; it does not grant administrator direct-forward permission.

Important implementation discrepancy found during source review: `requireScopedAdmin()` is documented in comments as admitting administrator, VPAA, VPAF, and OSAS, but its actual condition rejects every role other than administrator. Route handlers using this helper therefore currently return 403 to VPAA/VPAF/OSAS. The helper is used by user/office/complaint administrative routes. This annex reports the executing condition rather than the intended comment. Other admin handlers calling `requireAdmin()` are administrator-only by design.

F.3.5 API Request and Response Handling

Request parsing: handlers read query parameters from NextRequest/URL, path parameters from the Next.js params object, JSON via req.json(), or multipart form data for the same-origin student upload. Zod `safeParse` validates the principal request bodies. Validation failures commonly return HTTP 400 with a flattened error object; registration/verification handlers also catch malformed JSON and return a JSON 400 response. JSON parse exceptions are not normalized identically by every handler.

Authorization: handlers first resolve the current Auth.js session and then apply role and resource checks. Complaint handlers use ownership, assigned-office, assigned-staff, forwarding markers, active state, and office-head designation. Admin guard failures return 401 or 403 JSON. Most protected API requests rejected by the Next.js proxy also receive HTTP 403 JSON; route handlers still enforce their own decisive checks.

Persistence and side effects: Mongoose reads/writes user, complaint, timeline, assignment, notification, generated-report, and configuration collections. Workflow mutations record audit/timeline entries and notifications where implemented. Formal forwarding and resolution decisions use MongoDB transactions and re-check pending/current state before commit. Status changes use an explicit transition service; some transitions have dedicated routes and are rejected by the generic PATCH handler.

External services: verification, password/email-change, and workflow email are delivered through Resend. Attachment uploads/downloads and generated report files use Cloudflare R2. If R2 is not configured, relevant upload/report endpoints return 501. The cron SLA route requires a shared secret and returns checked/escalated/warned counts; it is also configured in vercel.json as a scheduled GET.

Error responses are route-specific rather than wrapped by one global JSON error envelope. Common implemented statuses include 400 for invalid request/state, 401 for missing authentication, 403 for denied roles/resource scope, 404 for absent resources, 409 for conflicts/stale decisions, 422 for complaint-routing/SLA prerequisites, 429 for rate limits, 500 for selected integration failures, 501 for unavailable storage configuration, and 503 for selected email/rate-limit/OSAS dependencies. The handler inventory and detailed tables identify the status behavior verified for each major endpoint.